

## HDOC DAY-34 ASSIGNMENT

### Verification of Distributive Properties of Matrix Multiplication

#### 1. Problem Understanding

The task is to write a menu-driven C program using separate functions and variable-length 2D arrays to verify two distributive properties of matrix multiplication.

- Left distributive property:  $A*(B+C) = (A*B) + (A*C)$ , where A is  $m \times n$  and B and C are  $n \times p$ .
- Right distributive property:  $(A+B)*C = (A*C) + (B*C)$ , where A and B are  $m \times n$  and C is  $n \times p$ .
- For each operation, the program must display the matrices obtained on both sides of the equation and state whether the selected property is verified. Both result matrices have dimensions  $m \times p$ .

#### 2. Standard Test Cases

The test cases include every input required by the selected operation.

| TC  | Choice | m | n | p | Matrix A      | Matrix B        | Matrix C        | Result   |
|-----|--------|---|---|---|---------------|-----------------|-----------------|----------|
| TC1 | 1      | 2 | 2 | 2 | 1 2 / 3 4     | 5 6 / 7 8       | 1 0 / 0 1       | Verified |
| TC2 | 1      | 2 | 3 | 2 | 1 2 3 / 4 5 6 | 1 0 / 2 1 / 3 2 | 2 1 / 1 2 / 0 1 | Verified |
| TC3 | 2      | 2 | 2 | 2 | 1 2 / 3 4     | 5 6 / 7 8       | 1 0 / 0 1       | Verified |
| TC4 | 2      | 2 | 3 | 2 | 1 2 3 / 4 5 6 | 2 1 0 / 1 2 3   | 1 2 / 0 1 / 2 0 | Verified |

#### 3. Algorithm

##### A. Common Menu Logic

1. Start.
2. Display the menu options.
3. Read the user's choice.
4. Use switch-case to select the required operation.
5. Call the corresponding function.
6. If the choice is 3, exit the program.
7. If the choice is invalid, display "Invalid choice".
8. Stop.

##### B. Left Distributive Property

1. Read m, n and p separately.
2. Declare  $A[m][n]$ ,  $B[n][p]$  and  $C[n][p]$ .
3. Read all elements of A, B and C.
4. Find  $B+C$  and store it in a temporary  $n \times p$  matrix.
5. Calculate  $A*(B+C)$  and store it as the left-hand side matrix.
6. Calculate  $A*B$  and  $A*C$  separately.
7. Add  $A*B$  and  $A*C$  to obtain the right-hand side matrix.
8. Display both  $m \times p$  result matrices.
9. Compare corresponding elements of both result matrices.
10. If all elements are equal, display that the left distributive property is verified; otherwise display that it is not verified.

##### C. Right Distributive Property

1. Read m, n and p separately.
2. Declare A[m][n], B[m][n] and C[n][p].
3. Read all elements of A, B and C.
4. Find A+B and store it in a temporary m x n matrix.
5. Calculate (A+B)\*C and store it as the left-hand side matrix.
6. Calculate A\*C and B\*C separately.
7. Add A\*C and B\*C to obtain the right-hand side matrix.
8. Display both m x p result matrices.
9. Compare corresponding elements of both result matrices.
10. If all elements are equal, display that the right distributive property is verified; otherwise display that it is not verified.

#### 4. C Program

```
#include <stdio.h>

void leftDistributive()
{
    int m, n, p, i, j, k;
    printf("Enter rows of A (m): ");
    scanf("%d", &m);
    printf("Enter columns of A / rows of B and C (n): ");
    scanf("%d", &n);
    printf("Enter columns of B and C (p): ");
    scanf("%d", &p);
    int A[m][n], B[n][p], C[n][p];
    int sum[n][p], lhs[m][p], AB[m][p], AC[m][p], rhs[m][p];
    printf("Enter matrix A:\n");
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &A[i][j]);
    printf("Enter matrix B:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < p; j++)
            scanf("%d", &B[i][j]);
    printf("Enter matrix C:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < p; j++)
```

```

        scanf("%d", &C[i][j]);
for (i = 0; i < n; i++)
    for (j = 0; j < p; j++)
        sum[i][j] = B[i][j] + C[i][j];
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        lhs[i][j] = 0;
        AB[i][j] = 0;
        AC[i][j] = 0;
        for (k = 0; k < n; k++)
        {
            lhs[i][j] += A[i][k] * sum[k][j];
            AB[i][j] += A[i][k] * B[k][j];
            AC[i][j] += A[i][k] * C[k][j];
        }
        rhs[i][j] = AB[i][j] + AC[i][j];
    }
}
printf("A*(B+C):\n");
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
        printf("%d ", lhs[i][j]);
    printf("\n");
}
printf("(A*B)+(A*C):\n");
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
        printf("%d ", rhs[i][j]);

```

```

    printf("\n");
}
int flag = 1;
for (i = 0; i < m; i++)
    for (j = 0; j < p; j++)
        if (lhs[i][j] != rhs[i][j])
            flag = 0;
if (flag == 1)
    printf("Left distributive property is verified.\n");
else
    printf("Left distributive property is not verified.\n");
}
void rightDistributive()
{
    int m, n, p, i, j, k;
    printf("Enter rows of A and B (m): ");
    scanf("%d", &m);
    printf("Enter columns of A and B / rows of C (n): ");
    scanf("%d", &n);
    printf("Enter columns of C (p): ");
    scanf("%d", &p);
    int A[m][n], B[m][n], C[n][p];
    int sum[m][n], lhs[m][p], AC[m][p], BC[m][p], rhs[m][p];
    printf("Enter matrix A:\n");
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &A[i][j]);
    printf("Enter matrix B:\n");
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &B[i][j]);
    printf("Enter matrix C:\n");

```

```

for (i = 0; i < n; i++)
    for (j = 0; j < p; j++)
        scanf("%d", &C[i][j]);
for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
        sum[i][j] = A[i][j] + B[i][j];
for (i = 0; i < m; i++) {
    for (j = 0; j < p; j++)
    {
        lhs[i][j] = 0;
        AC[i][j] = 0;
        BC[i][j] = 0;
        for (k = 0; k < n; k++) {
            lhs[i][j] += sum[i][k] * C[k][j];
            AC[i][j] += A[i][k] * C[k][j];
            BC[i][j] += B[i][k] * C[k][j];
        }
        rhs[i][j] = AC[i][j] + BC[i][j];
    }
}
printf("(A+B)*C:\n");
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
        printf("%d ", lhs[i][j]);
    printf("\n");
}
printf("(A*C)+(B*C):\n");
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
        printf("%d ", rhs[i][j]);

```

```

        printf("\n");
    }
    int flag = 1;
    for (i = 0; i < m; i++)
        for (j = 0; j < p; j++)
            if (lhs[i][j] != rhs[i][j])
                flag = 0;
    if (flag == 1)
        printf("Right distributive property is verified.\n");
    else
        printf("Right distributive property is not verified.\n");
}

int main()
{
    int choice;
    do {
        printf("\nMENU\n");
        printf("1. Verify left distributive property\n");
        printf("2. Verify right distributive property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                leftDistributive();
                break;
            case 2:
                rightDistributive();
                break;
            case 3:
                printf("Exiting program.\n");
                break;
        }
    } while (choice != 3);
}

```

default:

```
printf("Invalid choice.\n");
```

```
}
```

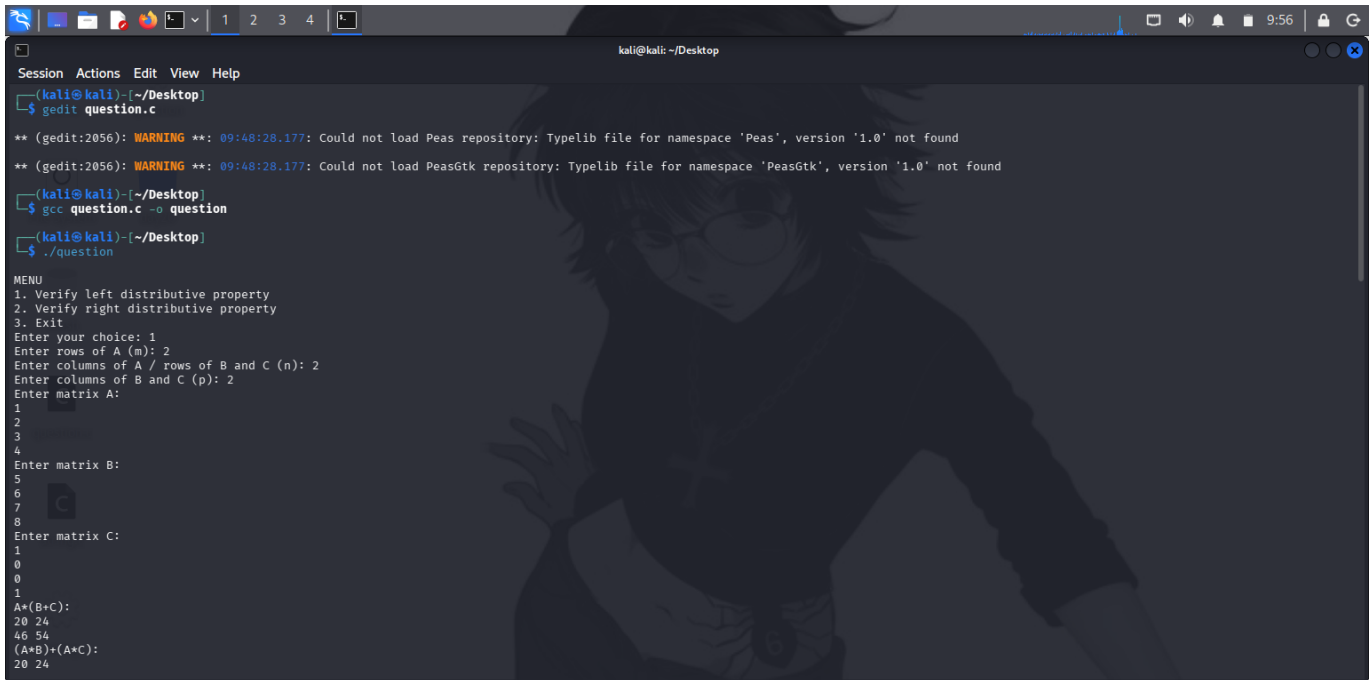
```
}
```

```
while (choice != 3);
```

```
return 0;
```

```
}
```

## 5. Compilation and Execution Evidence



```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)~/Desktop
$ gedit question.c

** (gedit:2056): WARNING **: 09:48:28.177: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:2056): WARNING **: 09:48:28.177: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~/Desktop
$ gcc question.c -o question

(kali@kali)~/Desktop
$ ./question

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 1
Enter rows of A (m): 2
Enter columns of A / rows of B and C (n): 2
Enter columns of B and C (p): 2
Enter matrix A:
1
2
3
4
Enter matrix B:
5
6
7
8
Enter matrix C:
1
0
0
1
A*(B+C):
20 24
46 54
(A*B)+(A*C):
20 24
```

```
kali@kali: ~/Desktop

Session Actions Edit View Help
46 54
Left distributive property is verified.

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 1
Enter rows of A (m): 2
Enter columns of A / rows of B and C (n): 3
Enter columns of B and C (p): 2
Enter matrix A:
1 2 3 4 5 6
Enter matrix B:
1 0 2 1 3 2
Enter matrix C:
2 1 1 2 0 1
A*(B+C):
18 16
45 37
(A*B)+(A*C):
18 16
45 37
Left distributive property is verified.

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 2
Enter rows of A and B (m): 2
Enter columns of A and B / rows of C (n): 2
Enter columns of C (p): 2
Enter matrix A:
1 2 3 4
Enter matrix B:
5 6 7 8
Enter matrix C:
1 0 0 1
(A*B)*C:
18 16
45 37
Right distributive property is verified.

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 3
Exiting program.
```

```
kali@kali: ~/Desktop

Session Actions Edit View Help
1 0 0 1
(A*B)*C:
6 8
10 12
(A*C)+(B*C):
6 8
10 12
Right distributive property is verified.

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 2
Enter rows of A and B (m): 2
Enter columns of A and B / rows of C (n): 3
Enter columns of C (p): 2
Enter matrix A:
1 2 3 4 5 6
Enter matrix B:
2 1 0 1 2 3
Enter matrix C:
1 2 0 1 2 0
(A*B)*C:
9 9
23 17
(A*C)+(B*C):
9 9
23 17
Right distributive property is verified.

MENU
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 3
Exiting program.

(kali@kali)-[~/Desktop]
$
```

## 6. Verification of All Test Cases

| TC  | Choice | m | n | p | Inputs                                      | Actual Result           | Status |
|-----|--------|---|---|---|---------------------------------------------|-------------------------|--------|
| TC1 | 1      | 2 | 2 | 2 | A=1 2/3 4; B=5 6/7 8; C=1 0/0 1             | Left property verified  | Pass   |
| TC2 | 1      | 2 | 3 | 2 | A=1 2 3/4 5 6; B=1 0/2 1/3 2; C=2 1/1 2/0 1 | Left property verified  | Pass   |
| TC3 | 2      | 2 | 2 | 2 | A=1 2/3 4; B=5 6/7 8; C=1 0/0 1             | Right property verified | Pass   |
| TC4 | 2      | 2 | 3 | 2 | A=1 2 3/4 5 6; B=2 1 0/1 2 3; C=1 2/0 1/2 0 | Right property verified | Pass   |